

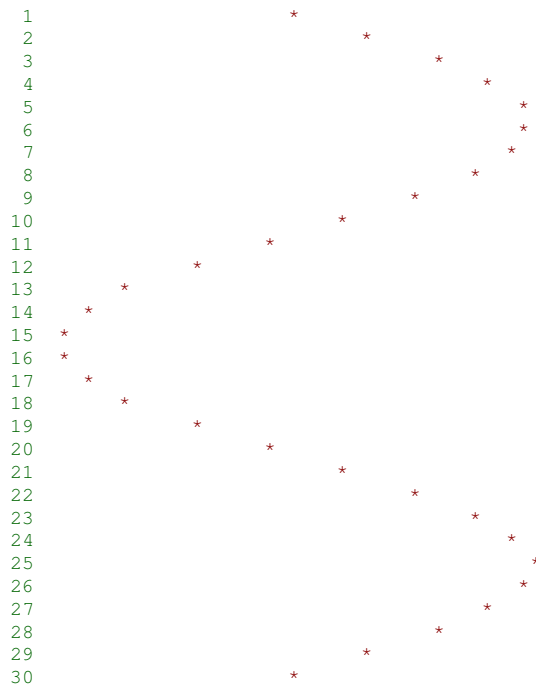
Programmeringsteknik för ingenjörer 1997

Föreläsning 8

Ändrad
16 Sep
16:59

funktioner

En sinuskurva. Grafiken är visserligen inte överväldigande.



Sinuskurvan plottas med hjälp av funktionen `sin` som finns i mattembiblioteket `math.h`.

Funktionerna i mattembiblioteket är av typen `double` -- flyttalstypen som använder fler värdesiffror än `float`, och som klarar större magnituder.

Den icke-matteintresserade kan ta lätt på beräkningsdetaljerna i programmet. Detta är ju inte en matematikkurs.

Alla vinklar är i radianer:

```
1  /*
2  Exempel 10, programmeringsteknik för ingenjörer 1997
3  Jan Tångring, 15/9 1997
4
5  Programmet plottar en sinuskurva.
6
7  Nyheter:
8      - sin
9      - putchar
10     - (matematik)
11  */
12
13 #include <stdio.h>
14 #include <math.h>
15
16 #define PI 3.141592654
17 #define WIDTH 40      /* Skärmens bredd */
18
19 void main()
20 {
```

```

21     int N = 30;           /* Antal plottpunkter */
22     double x0 = 0.0;      /* Startpunkt */
23     double xN = 3 * PI;   /* Slutpunkt */
24     double x, y, k;
25     double max, min;      /* Största och minsta värdet */
26     int i, j;             /* Slaskvariabler */
27
28
29     /* Hitta max- och minvärden i intervallet */
30
31     max = sin(x0);
32     min = max;
33     for (i = 0; i < N; i++) {
34         y = sin(x0 + i * (xN - x0) / (N - 1));
35         if (y > max)
36             max = y;
37         if (y < min)
38             min = y;
39     }
40
41
42     /* Plotta */
43
44     k = (WIDTH - 1) / (max - min);
45     for (i = 0; i < N; i++) {
46         y = sin(x0 + i * (xN - x0) / (N - 1));
47         for (j = 1; j <= k * (y - min); j++)
48             putchar(' ');
49         putchar('*');
50         putchar('\n');
51     }
52 }

```

Huvudsakligen handlar beräkningarna i programmet om att räkna ut hur många blanktecken ($0 \text{--} (\text{WIDTH}-1)$) som ska skrivas ut före stjärnan.

■ Själva beräkningen av *sinus* tar inte mycket plats. Den görs med ett funktionsanrop på rad 34 (och så en gång till på rad 46).

Parametern till `sin` är ett stort aritmetiskt uttryck som beräknar x -värdet att ta sinus på. Vi ska beräkna sinus i N punkter mellan x_0 och x_N .

■ Jag beräknar alla funktionsvärden två gånger. Den första gången (rad 31--39) för att hitta det största och minsta funktionsvärdet i intervallet.

Visserligen vet jag redan gränserna för funktionen *sinus* ($-1 \text{--} +1$). Men jag har skrivit programmet *generellt*. Man ska kunna byta ut *sinus* mot vilken funktion som helst.

■ Variablerna `min` och `max` innehåller de största respektive minsta funktionsvärden som stötts på. De initieras bägge två till `sin(x0)`.

■ Den andra gången jag räknar ut funktionsvärdena (rad 44--51) utförs också själva plottningen. På rad 47--48 finns en `for`-loop som skriver ut rätt antal blanktecken.

■ Stjärnan skrivs ut på rad 49. Följd av ny rad på rad 50.

`putchar`

Jag använder funktionen `putchar` för att skriva ut tecken.

Satsen `putchar(ch)` betyder (ungefär) detsamma som `printf("%c", ch)`

Heltal och flyttal i samma uttryck

På rad 34, bland annat, blandar jag heltal med flyttal. Detta är tillåtet.

För en gångs skull får vi hjälp av kompilatorn. Den omvandlar "uppåt" om det behövs. Den omvandlar automatiskt heltal till flyttal. Och flyttal av typen `float` till flyttal av typen `double`.

Typomvandling

Om det behövs kan man säga till om typomvandling. Antag att följande deklarationer gäller:

```
int i = 14, j = 5, k;
double x;
```

Kommentarerna talar om vad uttrycken nedan har för värde:

```
k = i / j;           /* k = 2, heltalsdivision */
x = i / j;           /* x = 2.0, omvandling efteråt */
x = i / (1.0 * j);   /* x = 2.8, för 1.0 * j == 5.0 */
x = i / (float) j;   /* x = 2.8, för (float) j == 5.0 */
```

Akta dig för att skriva

```
i = x;
```

För då vet man aldrig vad som kan hända. Skriv hellre

```
i = (int) x;   /* i = 2 */
```

Mattebiblioteket `math.h`

Mattebiblioteket innehåller bland annat följande funktioner:

- `sin, cos, tan`
- `asin, acos, atan` -- *arcus sinus*, etc
- `sinh, cosh, tanh` -- *sinus hyperbolicus*, etc
- `exp(x)` = e^x "upphöjt till" x
- `log(x)` = $\ln(x)$ -- naturliga logaritmen
- `log10(x)` = $\log(x)$ -- tiologaritmen
- `sqrt(x)` = "roten ur" x
- `ceil(x)` = x avrundat uppåt (`ceil(x) >= x`)
- `floor(x)` = x avrundat neråt (`floor(x) <= x`)
- `fabs(x)` = $|x|$

! Samtliga funktioner ovan arbetar på typen `double`. Även `ceil` och `floor`. Ta inte emot returvärdet som ett heltal! Ge dem inte heltal som parametrar! Effekterna är svårförutsägbara! !

Nu ska vi göra en egen funktion!

Våra program börjar bli stora och oöverskådliga.

Man kan dela upp sina program i mindre bitar. Det ger bättre ordning och överskådlighet.

Vi återvänder till ett program från föreläsning 4.
Programmet beräknar maximum av två tal:

```

1  #include <stdio.h>
2
3  main()
4  {
5      int i, j;
6
7      printf("Mata in två tal: ");
8      scanf("%d%d", &i, &j);
9
10     if (i > j)
11         printf("%d är störst. \n", i);
12     else
13         printf("%d är störst. \n", j);
14 }
```

Vi stoppar in själva max-beräkningen i en funktion:

```

1  #include <stdio.h>
2
3  int max(int i, int j)
4  {
5      if (i > j)
6          return i;
7      else
8          return j;
9  }
10
11
12 main()
13 {
14     int x, y;
15
16     printf("Mata in två tal: ");
17     scanf("%d%d", &x, &y);
18
19     printf("%d är störst. \n", max(x, y));
20 }
```

- Rad 19 -- `max(x, y)` -- är ett *funktionsanrop*. Värdena `x` och `y` kallas *parametrar*.
- Rad 3--9 är en *funktionsdefinition*. Variablerna `i` och `j` kallas (*formella*) *parametrar*. De är bägge av typen `int`.
- Början av rad 3 anger att funktionen ska returnera en `int`.
- På rad 6 eller 8 avbryts funktionen. Nyckelordet är `return`. Samtidigt anges vilket värde som ska returneras till anropet.
- `main` är också en funktion! Att utföra ett C-program är att anropa funktionen som heter `main`. Vi återkommer till detta.

Var noga med att parametrarna i anropet är av rätt typ!

! Det är inte säkert att kompilatorn varnar! !

Nästa lektion

